

Por que usamos banco relacional?

Por que uma API REST usa HTTP?

O que realmente é um endpoint?

Por que precisamos de autenticação e autorização?

JWT é realmente seguro?

Por que usamos Docker?

O que um container resolve?

O que acontece quando faço git push?

Como o DNS encontra um site?

Como HTTPS funciona?

O que acontece quando digito google.com no navegador?

Como um navegador transforma HTML em uma página?

Por que React usa componentes?

O que o TypeScript realmente faz?

Por que JavaScript é single-threaded?

Como o Node consegue lidar com várias requisições?

O que é uma race condition?

O que é uma transação no banco?

Por que índices deixam consultas mais rápidas?

O que acontece quando uma aplicação recebe 100 mil requisições?

O que realmente acontece quando uma função é chamada?

O que é uma abstração?

Quando uma abstração começa a ser exagerada?

O que torna um código legível?

O que torna um código difícil de manter?

Quando devo criar uma função separada?

Quando uma função está grande demais?

O que caracteriza código duplicado?

Por que DRY pode ser usado de maneira exagerada?

O que é acoplamento?

O que é coesão?

O que significa "baixo acoplamento e alta coesão"?

Por que algumas soluções simples são melhores que arquiteturas complexas?

Como identificar complexidade desnecessária?

O que é dívida técnica?

Como uma decisão técnica ruim se transforma em dívida técnica?

O que é refatoração?

Refatorar muda o comportamento do programa?

Como saber se uma refatoração foi bem-sucedida?

O que significa escrever código "idiomático"?

O que realmente significa complexidade $O(n)$?

Por que um algoritmo $O(n^2)$ pode ser mais rápido que um $O(n)$?

Quando uma busca linear é suficiente?

Como funciona uma busca binária?

Por que a busca binária exige uma estrutura ordenada?

Como uma tabela hash encontra um elemento?

O que acontece quando ocorre uma colisão em uma hash table?

Quando usar array e quando usar linked list?

Por que árvores existem?

O que torna uma árvore balanceada?

Como funciona uma árvore binária?

Como funciona um heap?

Para que serve uma fila de prioridade?

Como um sistema de rotas encontra um caminho?

Como funciona Dijkstra?

O que diferencia BFS de DFS?

Onde algoritmos de grafos aparecem em sistemas reais?

O que acontece quando digito uma URL no navegador?

Como o navegador descobre o IP de um domínio?

O que acontece depois que o IP é encontrado?

O que exatamente é uma conexão TCP?

O que acontece durante um handshake?

Por que existe HTTP?

Qual a diferença real entre HTTP e HTTPS?

O que acontece quando uma página faz fetch()?

Como cookies funcionam?

O que é uma sessão?

Como um servidor sabe quem sou?

O que é CORS?

Por que CORS existe?

O que é um preflight request?

O que é cache HTTP?

Quem decide se uma resposta pode ser armazenada em cache?

O que é um CDN?

Por que um CDN deixa um site mais rápido?

O que é um reverse proxy?

O que acontece entre o navegador e o servidor quando acesso uma aplicação?

O que realmente é um servidor?

O que significa "escutar uma porta"?

O que acontece quando duas requisições chegam simultaneamente?

Como o Node.js consegue lidar com várias requisições?

O que é programação assíncrona?

O que é uma Promise?

O que realmente acontece com async/await?

O que é uma thread?

O que é concorrência?

Concorrência e paralelismo são a mesma coisa?

O que é um processo?

Processo e thread são diferentes em quê?

O que acontece quando uma aplicação fica sem memória?

O que é um memory leak?

Como uma API deve tratar erros?

O que diferencia erro de negócio de erro técnico?

Onde deveria ficar a validação?

Onde deveria ficar a regra de negócio?

Controller deveria conter regra de negócio?

Por que existem Services?

O que é Repository Pattern?

Quando criar uma camada nova piora o projeto?

Por que bancos relacionais existem?

Por que não simplesmente guardar tudo em JSON?

O que realmente significa "relacional"?

O que é uma chave primária?

Por que uma tabela precisa de uma chave primária?

O que é uma foreign key?

Por que normalizamos bancos?

Quando normalização demais pode ser ruim?

O que acontece quando fazemos um JOIN?

Como o banco encontra uma linha específica?

O que é um índice?

Por que um índice acelera uma consulta?

Por que não colocar índice em todas as colunas?

O que acontece quando fazemos SELECT *?

O que é uma transação?

O que significa ACID?

O que acontece se duas pessoas alterarem o mesmo registro?

O que é race condition em banco?

O que é deadlock?

O que significa isolamento de transação?

Por que bancos podem ficar lentos?

Como eu investigaria uma query lenta?

O que realmente significa autenticar?

Autenticação e autorização são diferentes como?

Por que senhas não devem ser armazenadas diretamente?

O que é hashing?

Hash e criptografia são a mesma coisa?

O que é salt?

Como funciona um login?

O que acontece depois que faço login?

Como um servidor sabe que estou autenticado?

O que é JWT?

JWT é criptografia?

O que acontece se alguém roubar meu token?

O que é OAuth?

O que é SQL Injection?

Como SQL Injection acontece?

O que é XSS?

O que é CSRF?

O que é rate limiting?

Por que APIs precisam de rate limiting?

O que significa princípio do menor privilégio?

O que deveria acontecer se um usuário tentar acessar o recurso de outro?

Por que testar software?

O que realmente significa "um teste passou"?

O que é teste unitário?

O que é teste de integração?

O que é teste end-to-end?

Qual a diferença entre eles?

O que exatamente eu deveria testar?

O que não deveria testar?

O que é um mock?

Por que mocks podem ser perigosos?

Como testar uma API?

Como testar uma função que depende de banco?

Como testar casos extremos?

Como descobrir quais casos extremos existem?

O que é regressão?

Por que um teste pode ficar desatualizado?

Como saber se minha suíte de testes é boa?

O que realmente é um servidor?

O que é uma máquina virtual?

O que é um container?

Container e máquina virtual são diferentes como?

Por que Docker existe?

O que acontece quando executo docker run?

O que é uma imagem Docker?

O que é Docker Compose?

O que significa CI?

O que significa CD?

O que acontece quando faço git push e um pipeline começa?

Como uma aplicação chega até produção?

O que é deployment?

O que é rollback?

O que acontece quando uma versão nova quebra em produção?

O que é load balancing?

O que é escalabilidade?

Escalar verticalmente e horizontalmente é diferente como?

O que é alta disponibilidade?

O que é observabilidade?

O que é um IP?

Por que existem IPv4 e IPv6?

O que é uma porta?

Por que HTTP normalmente usa a porta 80?

Por que HTTPS usa 443?

O que é DNS?

O que é DHCP?

O que é NAT?

O que é firewall?

O que é TCP?

O que é UDP?

Quando UDP faz mais sentido?

Como um computador encontra outro computador na rede?

O que acontece quando dois dispositivos possuem o mesmo IP?

O que é localhost?

O que significa 127.0.0.1?

O que realmente acontece quando faço ping?

Por que separar frontend e backend?

Quando um monolito é suficiente?

Por que alguém usaria microsserviços?

Quais problemas microsserviços criam?

O que é Clean Architecture?

Qual problema ela tenta resolver?

O que é arquitetura em camadas?

O que é MVC?

MVC ainda faz sentido?

O que é Domain-Driven Design?

O que é SOLID?

O princípio SOLID mais fácil de entender é qual e por quê?

O que é Dependency Injection?

Por que Dependency Injection existe?

O que é inversão de dependência?

O que é Design Pattern?

Por que padrões podem ser usados em excesso?

Como decidir entre duas arquiteturas?

Como eu saberia que essa solução está errada?

Quais são os possíveis pontos de falha desse sistema?

O que aconteceria se o número de usuários aumentasse 100x?

O que acontece se o banco ficar indisponível?

O que acontece se uma requisição for enviada duas vezes?

O que acontece se uma resposta demorar 30 segundos?

O que acontece se dois usuários fizerem isso simultaneamente?

Como eu descobriria por que esse sistema está lento?

Como eu investigaria um bug que não consigo reproduzir?

Como eu faria essa solução ser mais fácil de testar?

Como eu faria essa solução ser mais fácil de trocar no futuro?

Qual é a solução mais simples que resolve o problema?

O que estou abstraindo sem necessidade?

Que suposições estou fazendo?

O que eu não sei sobre esse problema?

Como eu provaria que minha solução funciona?